

MODULE 2 · CORE WORKLOADS

Probes

How Kubernetes decides when to restart a Pod — and when to just stop sending it traffic

A crashed process and a stuck process look identical to Kubernetes

Left alone, Kubernetes only notices a container is unhealthy when the process **exits**. A web server that hangs — never crashes, never responds — looks perfectly Running while your app quietly stops working.

Probes let you tell Kubernetes what "healthy" means for your app, and what to do about it: restart it, hold back traffic, or just wait a little longer.

BY THE END YOU CAN

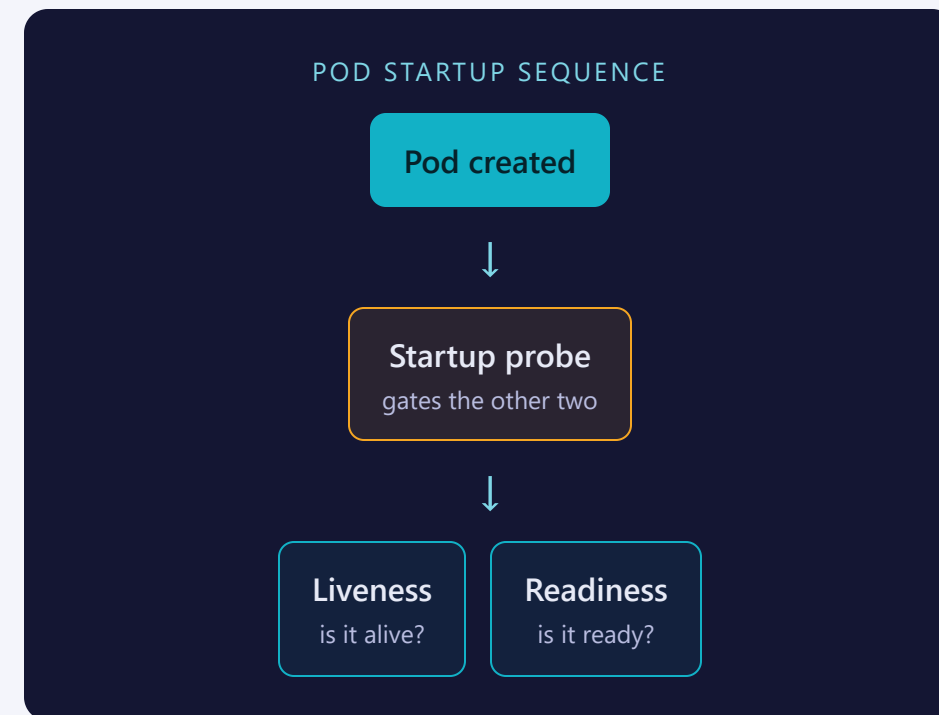
- Explain liveness vs readiness vs startup
- Configure httpGet, tcpSocket & exec probes
- Tune the key timing knobs
- Read a probe failure from Events

CONCEPT

Three probes, three different jobs

- **Startup** — has the app finished booting?
- **Liveness** — is it still working, or hung?
- **Readiness** — can it handle traffic right now?

All three run the same kind of check (httpGet / tcpSocket / exec) on a schedule — they only differ in what Kubernetes **does** with the result.



Same signal, opposite consequences

PROBE	QUESTION IT ANSWERS	ON FAILURE
Liveness	Is the app still working?	Container is killed & restarted
Readiness	Can it serve traffic right now?	Removed from Service endpoints — no restart
Startup	Has it finished starting?	Liveness & readiness stay disabled until it succeeds

MENTAL MODEL

Liveness answers "should this container exist?" Readiness answers "should traffic go here right now?" Two very different questions, two very different actions.

A probe is a check plus a schedule

```
apiVersion: v1
kind: Pod
metadata:
  name: probe-demo
  namespace: training
  labels:
    app: probe-demo
spec:
  containers:
  - name: app
    image: busybox:1.36
    command:
      - /bin/sh
      - -c
      - "echo 'Starting slow app...' && sleep 20 && touch /tmp/ready && echo 'App started!' && while true; do echo 'healthy' > /tmp/health; sleep 5; done"
    startupProbe:
      # gates liveness & readiness until it succeeds
      exec:
        command:
          - cat
          - /tmp/health
        initialDelaySeconds: 5
        periodSeconds: 5
        failureThreshold: 10 # up to 50s to boot
    livenessProbe:
      # restarts the container on failure
      exec:
        command:
          - cat
          - /tmp/health
        periodSeconds: 10
        failureThreshold: 3
    readinessProbe:
      # pulls from Service endpoints on failure
      exec:
        command:
          - cat
          - /tmp/ready
        initialDelaySeconds: 5
        periodSeconds: 5
        failureThreshold: 10
```

Probes rarely start imperative

IMPERATIVE — FAST, ONE-OFF

```
kubectl run web --image=nginx:1.25
```

`kubectl run` has no probe flags — nothing to set on the fly

DECLARATIVE — VERSIONABLE, REPEATABLE

```
kubectl apply -f pod-liveness.yaml
```

The probe block is part of the file, reviewed and re-appliable

THE BRIDGE TRICK

Generate the base spec, then hand-add the probe block it can't express:

```
$ kubectl run web --image=nginx:1.25 \  
  --dry-run=client -o yaml > pod.yaml # then add livenessProbe: by hand
```

This lab's manifest composes the startupProbe + livenessProbe from `pod-startup.yaml` with the readinessProbe timing from `pod-readiness.yaml`; the lab itself runs each as a separate Pod (`liveness-http`, `readiness-test`, `startup-test`).

SEE IT

Two failures, two very different outcomes

LIVENESS PROBE FAILS 3X

Pod running



kubelet kills the container



Restarted

RESTARTS++

READINESS PROBE FAILS

Pod running



Removed from Service endpoints



Still running

0/1 READY, no traffic

Three ways to ask the question

TYPE	HOW IT CHECKS	USED IN THIS LAB FOR
<code>httpGet</code>	GET request to a path; 2xx–3xx = healthy	liveness-http (nginx <code>/</code>)
<code>tcpSocket</code>	Opens a TCP connection to a port	non-HTTP servers, databases
<code>exec</code>	Runs a command inside the container; exit 0 = healthy	readiness-test, startup-test (<code>cat</code> a file)

SAME IDEA, DIFFERENT PLUMBING

Pick whichever check your app can already answer — a health endpoint, an open port, or a file/command a script can flip.

Where people trip

- **No readiness probe.** The Service sends traffic the instant the container starts, ready or not.
- **Liveness == readiness probe.** A slow dependency restarts the container instead of just pulling it from rotation.
- **No startup probe on slow apps.** Liveness kills it before it finishes booting.
- **Thresholds too tight or too loose.** Too tight → needless restarts; too loose → a hung app stays in rotation for minutes.

Commands to keep at hand

```
$ kubectl describe pod NAME           # Liveness/Readiness/Startup + events
$ kubectl get pod NAME -w             # watch READY column & RESTARTS
$ kubectl get pod NAME -o jsonpath='{.spec.containers[0].livenessProbe}'
$ kubectl exec NAME -- rm /tmp/ready  # simulate a readiness failure
$ kubectl get pod NAME -o jsonpath='{.status.containerStatuses[0].restartCount}'
```

Tip: a probe failure shows up as an Event long before RESTARTS ticks up — describe first.

RECAP

Probes in one breath

- Startup gates liveness & readiness until the app has booted
- Liveness failure → kubelet kills & restarts the container
- Readiness failure → Pod keeps running, just pulled from Service endpoints
- httpGet / tcpSocket / exec — three ways to ask "is it healthy?"

HANDS-ON

Now run Lab 2.5
`labs/2.5/`

Next: [2.6 — StatefulSets Basics](#)